Smart Lift Controller - Microarchitecture & RTL Design Journey

This document summarizes the architectural evolution and RTL engineering decisions taken during the development of the Smart Lift Controller, transitioning from initial conceptual abstractions to an optimized, hardware-realizable ASIC microarchitecture.

Executive Summary of Design Shifts

Design Metric	Initial Concept (Before)	Hardware Realization (After)	Engineering Rationale
System Model	Single-button predictive hall call	Directional Collective Sweep	Prevents skipped floors and accurately tracks intent.
I/O Overhead	Unconstrained pinout, redundant inputs	Strict 8-In / 8-Out Pin Interface	Fits physical package boundaries; eliminates phantom presses.
Motor Interface	Raw `motor_up` / `motor_down` signals	`motor_enable` + `motor_dir`	Prevents hardware short-circuits across bridge drivers.
Partitioning	Monolithic software-style script	Tri-module RTL Architecture	Isolates datapath, memory latching, and control logic.
Control Logic	5-State Machine with mode splits	3-State FSM with Parallel Lookahead	Eliminates redundant cycles and simplifies synthesis.

1. System Architecture & Intent

- **Initial Concept:**
    - Designed around a single hall button per floor across a 5-floor structure.

    - Relied on speculative software-style logic to predict user direction, which frequently caused lift skip-overs for passengers waiting to travel in the opposite direction.
- **Refined Architecture:**
    - Shifted to a standard **Directional Collective Architecture** equipped with dedicated Up and Down hall call buttons.
    - System now accurately latches directional requests, enabling a classic, deterministic elevator sweep algorithm that maximizes service efficiency.

2. Pinout Optimization & Hardware Constraints

- **Initial Concept:**
    - Inefficient pin utilization, passing internal state parameters (e.g., `curr_floor_no` and current `state`) as top-level inputs.
    - Included unnecessary and redundant control lines like `insider_dir`.
- **Refined Architecture:**
    - Streamlined top-level interface to fit an **8-Pin Input / 8-Pin Output** hardware boundary.
    - **Input Interface Improvements:** Dropped `insider_dir` and added a `req_valid` strobe line. This ensures resting zero states (`00`) are not registered as accidental phantom requests.
        - *Final Inputs:* `clk`, `rst_n`, `outsider_loc[1:0]`, `insider_dest[1:0]`, `outsider_dir`, `req_valid`
    - **Output Interface Improvements:** Redesigned motor control to use `motor_enable` and `motor_dir` flags rather than dual direction power lines, avoiding hardware short-circuits. Removed door-timer states to optimize single-cycle floor transitions.
        - *Final Outputs:* `motor_enable`, `motor_dir`, `curr_floor_out[1:0]`, `dir_up_led`, `dir_down_led`

3. RTL & Microarchitecture Partitioning

- **Initial Concept:**
    - Code structured like software, utilizing procedural `if/else` procedural chains attempting to "call" travel modules dynamically during execution.

- **Refined Architecture:**
    - Partitioned the top-level design into three discrete physical submodules to resolve routing congestion and prevent timing closure issues:
        - `request_manager` **(Memory Block):** Dedicated D-flip-flop array to reliably catch, latch, and clear single-cycle pulse button presses.
        - `floor_tracker` **(Datapath):** Isolated synchronous counter handling physical floor tracking and mathematical updates.
        - `lift_fsm` **(Control Unit):** Pure control state engine evaluating registered inputs against current datapath states.

4. FSM Optimization & Lookahead Logic

- **Initial Concept:**
    - A 5-state FSM that segregated interior car calls from exterior hall calls, creating priority deadlocks and bypassing waiting passengers.
- **Refined Architecture:**
    - Consolidated into a streamlined **3-State FSM** (`IDLE`, `MOVE_UP`, `MOVE_DOWN`).
    - Merged all request sources (`car_calls`, `hall_up`, `hall_down`) into a single consolidated global bitmask.
    - Implemented parallel combinational lookahead logic (`req_above`, `req_below`, `stop_here`) to directly drive motor enables and directional routing in a single clock cycle, avoiding nested iterative loops.